Природо-математическа гимнзия „Васил Друмев“

Велико Търново

Предмет

Програмиране на информационни системи

Проект

„Библиотечна система (Library) “

Изготвил:

Кристиана Мирославова Стоянова

Специалност: Софтуерни и хардуерни науки

Задал:

Мариана Русанова

Град Велико Търново

2026

Съдържание:

1. Цели и задачи на проекта

2. Проектиране на база данни – Library
   2.1 Таблици от базата данни
   2.2 Връзка с базата от данни
   2.3 ER диаграма

3. Реализация на приложението
   3.1 Разработка
   3.2 Основни елементи
   3.3 Изглед

4. Функционалност
   4.1 Основни класове и зависимости
   4.2 Ключови полета
   4.3 Зареждане на данни (LoadData)
   4.4 UI логика за RadioButtons
   4.5 Филтриране (LoadFilters, btnFilter_Click)
   4.6 Вмъкване (btnInsert_Click)
   4.7 Обновяване (btnUpdate_Click_1)
   4.8 Сортиране (btnSort_Click_1)
   4.9 Интеракция с DataGridView
   4.10 Грешки и поведение при изключения
   4.11 Кратък поток на данни (пример)

5. Заключение

1. Цели и задачи на проекта

Целта на проекта е създаване на **Windows Forms** приложение, което работи с релационна база данни **„Library**“ чрез Entity Framework.

Представлява формуляр за **CRUD** операции върху библиотечни данни (Books, Readers, Borrowings, Events) с помощта на Entity Framework (Database First).

Интерфейсът използва RadioButtons за избор на активна таблица, DataGridView за списък и няколко TextBox/Label за въвеждане/редакция, плюс бутони за Insert, Update, Filter, Sort и Exit.

Основните задачи на проекта са:

- Свързване на съществуваща база от данни с приложението
- Извличане и визуализация на данни
- Реализиране на CRUD операции (Insert, Update)
- Филтриране и сортиране на данни

2. Проектиране на база данни - Library

2.1 Таблици от базата данни

• **Readers** (Читатели): Таблицата съхранява информация за регистрираните читатели в библиотеката.

Основни полета:

id – уникален идентификатор

name – име на читателя

email – електронна поща

phone_number – телефонен номер

date_registration – дата на регистрация

• **Librarians** (Библиотекари): Съдържа данни за служителите на библиотеката.

Основни полета:

id – уникален идентификатор

name – име на библиотекаря

phone_number – телефонен номер

• **Events** (Събития): Описва събития, организирани в библиотеката.

Основни полета:

id – уникалн идентификатор

name – наименование

date – дата на провеждане

description – описание

librarian_id – библиотекар, отговорен за събитието; външен ключ за връзка с таблица Библиотекари

• **Events_readers** (Събития – Читатели): Свързваща таблица за реализиране на връзка много към много между читатели и събития.

Основни полета:

reader_id – външен ключ за връзка с таблица Читатели

event_id – външен ключ за връзка с таблица Събития

• **Authors** (Автори): Съдържа информация за авторите на книги.

Основни полета:

id – уникален идентификатор

name – име на автора

• **Categories** (Категории): Описва категориите, към които принадлежат книгите.

Основни полета:

id – уникален идентификатор

name – име на категорията

description – описание

• **Books** (Книги): Съдържа информация за наличните книги в библиотеката.

Основни полета:

id – уникален идентификатор

heading – заглавие

year – година на издаване

author_id – външен ключ за връзка с таблица Автори

category_id – външен ключ за връзка с таблица Категории

• **Borrowings** (Заемания): Описва процеса по заемане и връщане на книги.

Основни полета:

id – идентификатор

librarian_id – библиотекар

reader_id – читател

book_id – книга

date_got – дата на заемане

date_return – дата на връщане

status – статус (например „borrowed“, „returned“)

2.2 Връзка с базата от данни

За осъществяване на връзка между приложението и базата данни „Library“ е използвана технологията Entity Framework в среда на .NET Framework. Тя позволява работа с базата данни чрез обекти, без да е необходимо писане на SQL заявки. Първо се инсталира на NuGet пакет, за да може проектът да работи с Entity Framework.

Във Visual Studio се отваря проектът. Натиска се десен бутон върху проекта → Manage NuGet Packages. Търси се пакет: **EntityFramework**. Натиска се Install. След инсталацията проектът вече може да работи с база данни чрез **ORM** (Object-Relational Mapping). Създаване на модел (Model). Следващата стъпка е създаване на модел, който представлява връзката с базата данни. Моделът в проекта се казва: **LibraryEntities**, който е създаден чрез:

Десен бутон върху проекта

Add → New Item

Избира се ADO.NET Entity Data Model

EF Designer from database

Създава се връзка към базата данни (Connection String)

Избират се таблиците:

✅ Books

✅ Readers

✅ Borrowings

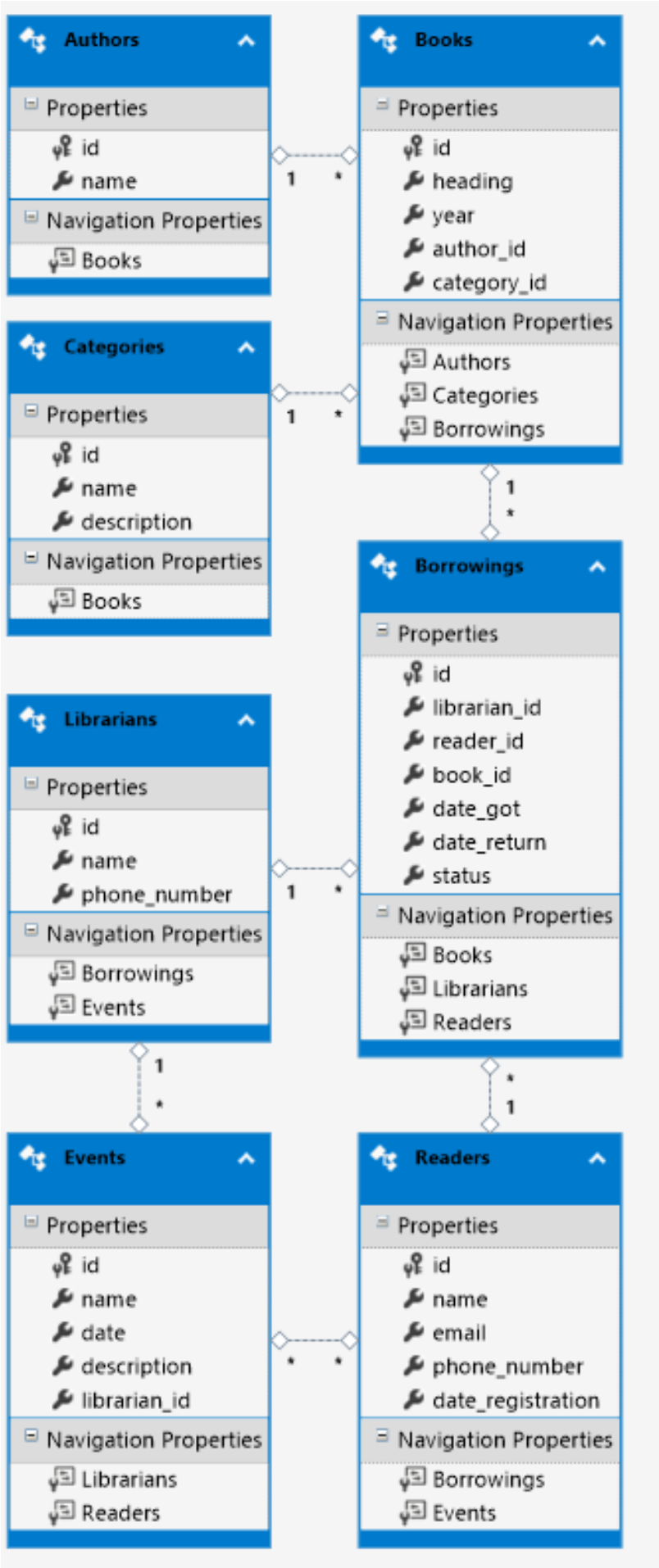
✅ Events

При създаване на модела автоматично се генерира връзка в**: App.config**. Това казва на приложението: къде се намира базата, как да се свърже с нея.

```
  <add name="LibraryEntities"
connectionString="metadata=res://*/Library.csdl|res://*/Library.ssdl|res://*/Library.m
sl;provider=System.Data.SqlClient;provider connection string=&quot;data
source=DESKTOP-214LU59\SQLEXPRESS;initial catalog=Library;integrated
security=True;encrypt=False;MultipleActiveResultSets=True;App=EntityFramework&quot;"
providerName="System.Data.EntityClient" /></connectionStrings>
```

След създаването на модела автоматично се генерират класове за всяка таблица.

2.3 Създаване на ER диаграма



3. Реализация на приложението

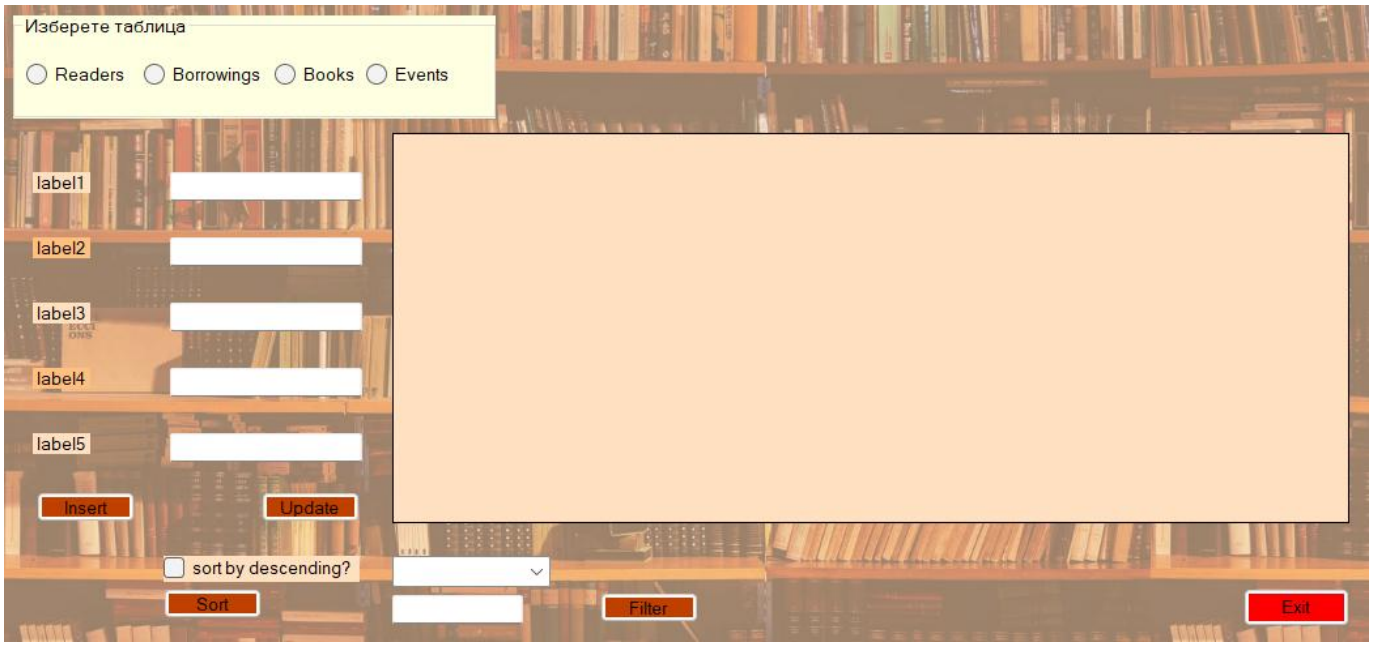
3.1 Приложението е разработено с

- C#
- Windows Forms
- Entity Framework

3.2 Основни елементи

- **DataGridView** – показва данните
- **RadioButton-и** – избор на таблица
- **TextBox-ове** – въвеждане на данни
- **ComboBox** – избор на филтър
- **Бутони** – действия

3.3 Изглед



4. Функционалност

4.1 Основни класове и зависимости

- Използвани библиотеки: System, System.Linq, System.Windows.Forms, System.Data.Entity.

4.2 Ключови полета

- private int editId = -1: държи id на текущо селектиран запис за Update/Delete операции.

4.3 Зареждане на данни (LoadData)

- Създава нов LibraryEntities контекст в using-блок.

- Изключва ProxyCreationEnabled и LazyLoadingEnabled, за да се избегнат lazy-load навън от живота на контекста.
- В зависимост от избрания RadioButton прави проекция (Select) към анонимни типове, включва свързани имена чрез Include (Authors, Categories, Librarians) и присвоява резултата на dataGridView1.DataSource.
- При Borrowings скрива помощните numeric id колони (ReaderId, BookId) в DataGridView, като ги запазва в DataSource за вътрешна употреба.
- Нулира селекцията на DataGridView и editId след зареждане.

4.4 UI логика за RadioButtons

- rbBooks/rbReaders/rbBorrowings/rbEvents_CheckedChanged: презареждат данни и филтри, конфигурират видимост и текст на label/textbox контролите според активния режим.

4.5 Филтриране (LoadFilters, btnFilter_Click)

- LoadFilters попълва cbFilter с подходящи полета според текущия режим.

btnFilter_Click използва стойността от cbFilter и txtFilter, създава IQueryable заявка и прилага Where защитно (вкл. TryParse за числа и дати). За Date полета използва DbFunctions.TruncateTime за сравнение по дата.

- След филтриране присвоява резултат към DataGridView и при нужда скрива id колони.

4.6 Вмъкване (btnInsert_Click)

- В зависимост от ел. режим създава нов EF обект (Books/Readers/Borrowings/Events) със стойности от текстовите полета и извиква context.SaveChanges().
- За Borrowings използва парсинг на id полета; date_return се подава като nullable чрез DateTime.TryParse.

4.7 Обновяване (btnUpdate_Click_1)

- Извлича съответния обект чрез context.Find(editId), обновява полетата от текстбоксове и извиква SaveChanges().

4.8 Сортиране (btnSort_Click_1)

- Чете cbFilter (sortKey) и по него прилага OrderBy/OrderByDescending върху IQueryable. За сортиране по навигационни свойства използва Include.
- Поддържа опционален cbDescending (ако е наличен) за обратен ред.

4.9 Интеракция с DataGridView

- dataGridView1_CellClick: задава editId от колоната "Id" на селектирания ред и попълва текстовите полета с показаните стойности (за Borrowings — използва ReaderId/BookId, въпреки че колоните са скрити).

4.10 Грешки и поведение при изключения

- DataGridView.DataError обработчик потиска стандартния диалог (e.ThrowException = false; e.Cancel = true).
- btnSort_Click_1 има общ try/catch, който показва MessageBox при грешки.

- Въвеждането използва много int.Parse/DateTime.Parse, което изисква валидиране — при невалиден вход ще хвърля FormatException.

4.11 Кратък поток на данни (пример)

- Потребител избира режим (Books) -> rbBooks_CheckedChanged задава UI и извиква LoadData().
- LoadData() чете Books с Include(Authors, Categories) и попълва DataGridView.
- Потребител селектира ред -> dataGridView1_CellClick задава editId и попълва текстовете.
- Потребител променя полета и натиска Update -> btnUpdate_Click_1 намира запис с editId, обновява полетата и SaveChanges(); след това LoadData() обновява списъка.

5. Заключение

В този проект беше разработена библиотечна система, която:

- Работи с реална база данни
- Използва релации между таблици
- Поддържа: добавяне, редактиране, филтриране, сортиране
- Използва Entity Framework за лесна работа с данни

Проектът демонстрира успешно:

- работа с релационни бази
- LINQ заявки
- създаване на потребителски интерфейс